The Ark AI

Lightning-Powered AI Services Platform

Software Documentation for Copyright Registration

Copyright Work Title:	The Ark AI - Lightning-Powered AI Services Platform
Software Version:	1.0.0
Software Developers:	The Navigator
Software Owners:	The Navigator
Platform URL:	https://arknode.ai
Alternative URL:	https://thenode.it.com
Date:	March 2026

Abstract

The Ark AI is a web-based artificial intelligence services platform that enables users to access over 120 AI-powered task types through a pay-per-task model using Bitcoin Lightning Network micropayments. The platform provides services including text summarization, code generation, translation, research, document analysis, image generation, voice synthesis (including African languages such as Swahili and Amharic), file conversion (170+ format conversions), and many more professional AI tools.

Built using modern web technologies including Python (FastAPI/Uvicorn), JavaScript, HTML5/CSS3, and integrated with DeepInfra API (Qwen 3.5 large language model), OpenAI APIs (DALL-E for image generation, Whisper for transcription, TTS for voice), Phoenixd (Lightning Network node), and LNbits (Bitcoin wallet management). The backend leverages LibreOffice, ImageMagick, and FFmpeg for file conversion capabilities.

The platform runs as a cloud-hosted web application accessible through any modern web browser on desktop and mobile devices. It is deployed on a Linux (Ubuntu 24.04) virtual private server with Nginx as a reverse proxy, served over HTTPS with SSL/TLS encryption. The platform is accessible at https://arknode.ai and https://thenode.it.com.

1.0 Introduction

The Ark AI is an innovative AI services platform designed to democratize access to artificial intelligence by making it affordable through Bitcoin Lightning micropayments. Unlike traditional AI platforms that require monthly subscriptions or credit card payments, The Ark AI allows users to pay as little as 10 satoshis (approximately $0.007 USD) per task, making AI accessible to anyone with a Lightning wallet.

Key functionalities of the platform include:

- **AI Chat Interface -** Users interact with the AI through a conversational chat interface with conversation memory, supporting text input, voice input, and image uploads.
- **Task Categories -** Over 120 task types organized across categories: coding, writing, research, legal, medical, education, finance, real estate, marketing, document processing, and more.
- **File Converter -** A dedicated file conversion service supporting 170+ conversion types across 5 categories (documents, images, audio, video, spreadsheets) with instant processing.
- **Image Generation -** AI-powered image creation using DALL-E, accessible through the chat interface.
- **Voice Services -** Text-to-speech in multiple languages including African languages (Swahili, Amharic), plus audio transcription via Whisper.
- **Lightning Payments -** Instant micropayments via Bitcoin Lightning Network, with QR code invoices for easy mobile payment.
- **Free Trial System -** New users receive 3 free prompts to try the service before paying.
- **Wallet System -** Users can create in-app wallets, deposit funds via Lightning, and pay for tasks from their balance.
- **OpenAI-Compatible API -** Developers can integrate The Ark AI into their applications using the standard OpenAI API format at /v1/chat/completions.
- **PDF Document Generation -** Automatic generation of downloadable PDF documents for applicable task responses.

The platform is designed to be mobile-first, with a responsive interface that works seamlessly on

smartphones, tablets, and desktop browsers. All AI processing occurs server-side, requiring no special hardware or software from the user.

2.0 Description of The Ark AI Platform

2.1 Accessing The Ark AI

The Ark AI platform is accessible through the URL https://arknode.ai or via the alternative domain https://thenode.it.com. The platform loads directly in the browser with no account creation required. Users are presented with a clean, dark-themed chat interface with quick-action buttons for popular tasks. A screenshot of the home page is shown below.

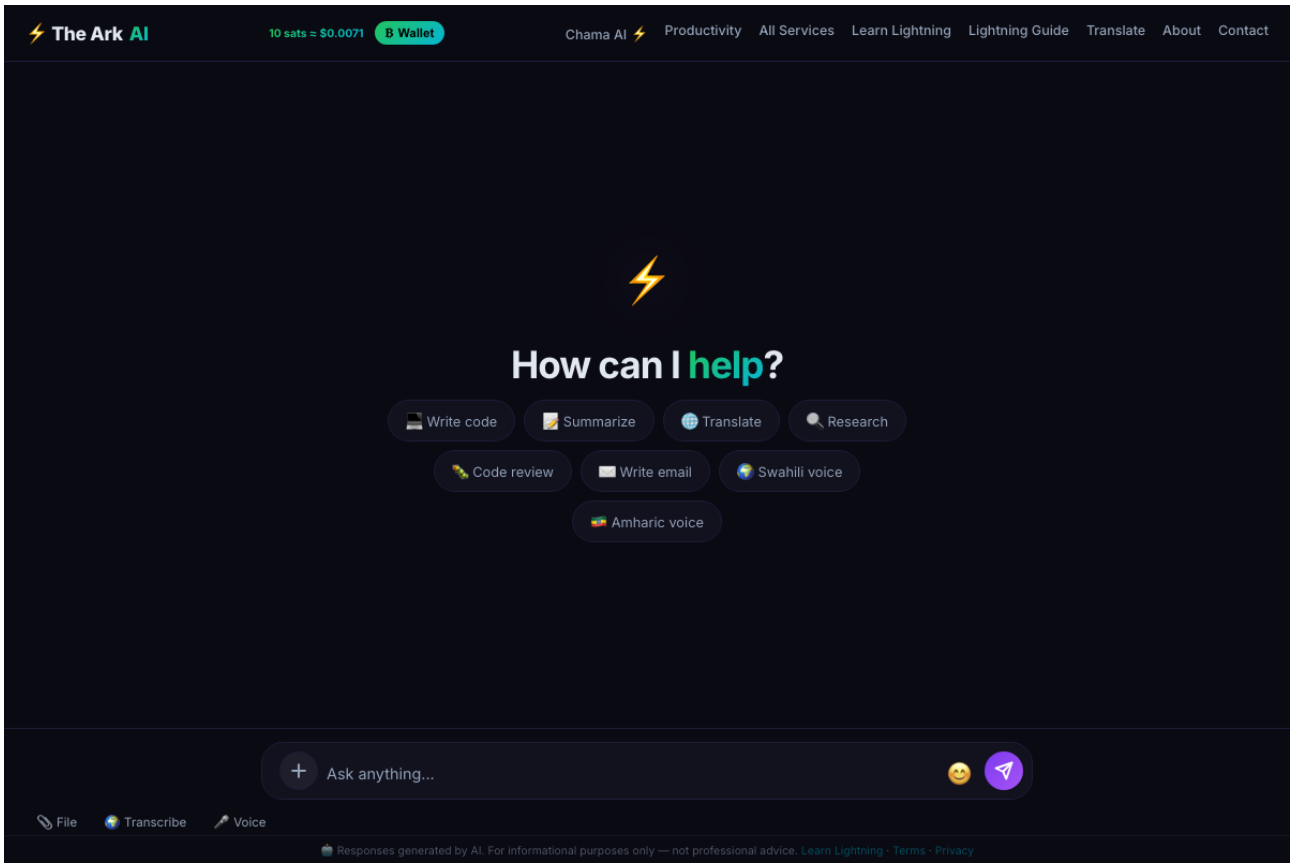

Figure 1: The Ark AI Home Page - Main chat interface with task quick-actions

2.2 Using the AI Chat Interface

Users interact with The Ark AI by typing or speaking their requests into the chat input field at the bottom of the screen. The interface supports:

- Text input via keyboard
- Voice input via microphone (browser speech recognition)
- Image uploads for visual analysis tasks
- File attachments for document processing
- Quick-action buttons for common tasks (Write code, Summarize, Translate, Research, etc.)

The AI processes requests using the Qwen 3.5 large language model (122 billion parameters) and streams responses in real-time. The chat interface maintains conversation context across messages within the same

session.

2.3 Lightning Payment System

When a user submits a task (after exhausting their 3 free prompts), the system generates a Bitcoin Lightning invoice displayed as a QR code. Users scan this QR code with any Lightning wallet to make an instant payment. Once payment is confirmed, the AI processes the task and streams the response. Pricing starts at 10 satoshis (~$0.007) for simple tasks and scales up for complex tasks like code generation (100 sats) and image generation (500 sats).

2.4 File Converter Service

The Ark AI includes a comprehensive file conversion service accessible at https://arknode.ai/convert/. This service supports 170+ conversion types across 5 categories: documents (PDF, DOCX, TXT, HTML, RTF, etc.), images (PNG, JPG, WebP, SVG, etc.), audio (MP3, WAV, FLAC, AAC, etc.), video (MP4, AVI, MOV, MKV, etc.), and spreadsheets (CSV, XLSX, ODS, etc.). Users upload a file, select the target format, and receive the converted file instantly.

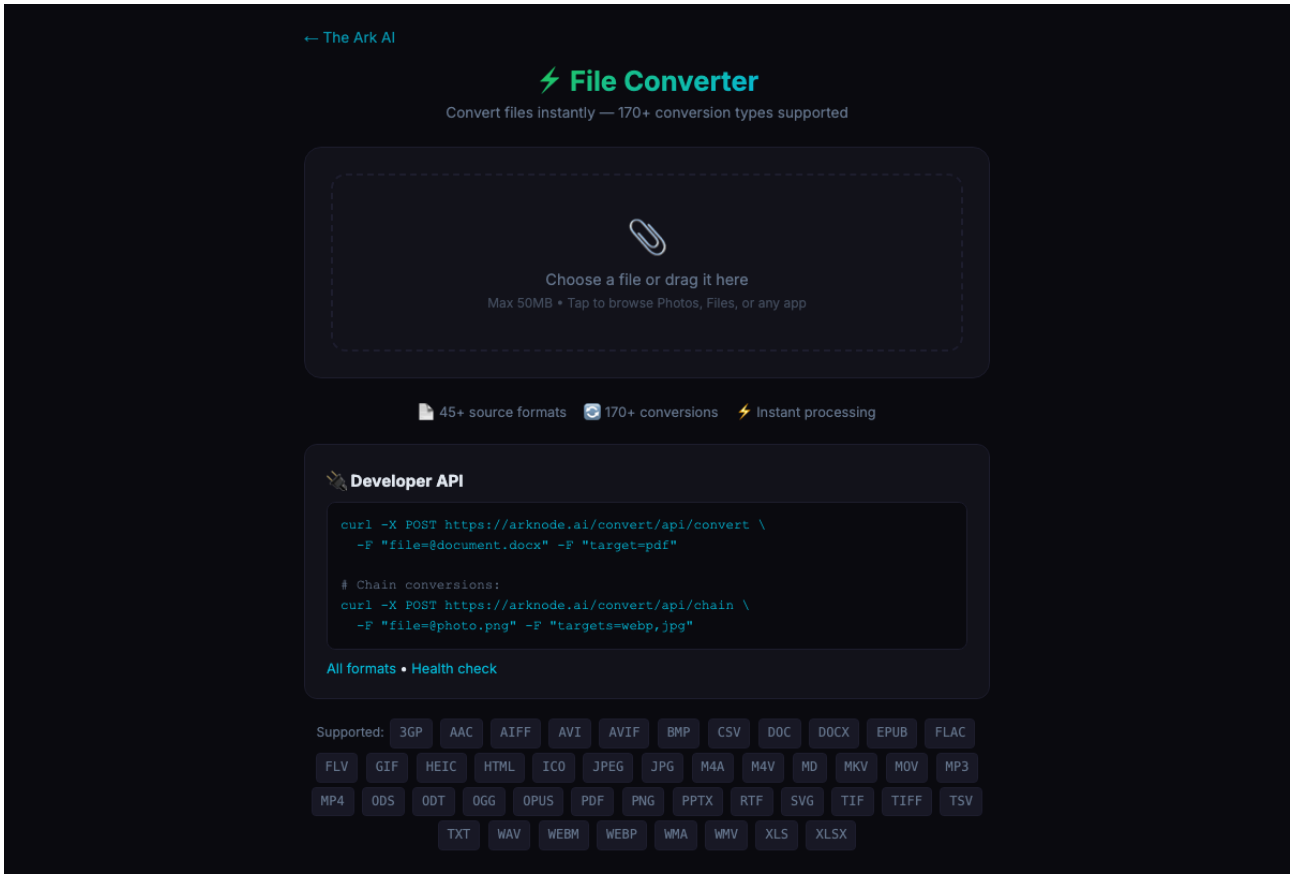

Figure 2: File Converter Interface - Upload and convert files between 170+ formats

2.5 Educational Content - Learn Lightning

The platform includes an educational page at https://arknode.ai/learn that explains Bitcoin Lightning Network technology, how micropayments work, why Lightning is suitable for AI services, and how to set up a Lightning wallet. This page serves both as user onboarding and as educational content about the underlying payment

technology.

⚡ The Ark AI

Chat Services Learn Lightning Guide About

⚡ What is Bitcoin Lightning?

A beginner-friendly guide to the payment network that makes AI services instant, cheap, and available to everyone on Earth.

⛈️ Bitcoin Lightning, Simply Explained

💡 The Simple Version

Bitcoin is like a bank vault — super secure, but slow and expensive for small payments. **Lightning** is like a tab at your favorite bar: you open a channel, make instant payments back and forth, and settle up on the main chain later.

🏗️ How It Was Built

Lightning is a **"Layer 2"** network built on top of Bitcoin. Think of Bitcoin as the foundation (Layer 1) and Lightning as the express lane on top. Transactions happen off-chain in payment channels, making them nearly instant and almost free — while still being backed by Bitcoin's security.

~10 min Bitcoin on-chain confirmation	< 1 sec Lightning payment
$0.50–$5+ Bitcoin on-chain fee	< $0.01 Lightning fee

⚙️ How Lightning Payments Work

When you pay with Lightning, your payment hops through a network of channels to reach the recipient — all in under a second.

🔐 It's Trustless & Secure

Lightning uses **cryptographic hash locks** — meaning your payment either fully completes or fully fails. No middlemen can steal your funds. It's peer-to-peer, just like Bitcoin itself, but 1000× faster.

🤖 Why Lightning is Perfect for AI

⚡ Instant

AI responses take seconds. Your payment should too. With Lightning, you pay and get results immediately — no waiting for bank confirmations or card processing.

💰 Micropayments

An AI query might cost 5–50 sats (fractions of a cent). Traditional payments can't handle amounts this small — minimum fees would be more than the service itself. Lightning was *designed* for micropayments.

🌍 Global, No Barriers

No credit card? No bank account? No problem. Lightning works for anyone with a smartphone, anywhere in the world. No KYC, no signup forms, no geographic restrictions.

🔒 Private

No email required. No personal data collected. Pay with sats, get your result, done. Your AI conversations stay between you and the model.

🔑 How The Ark Uses Lightning

The Ark uses the **L402 protocol** (formerly LSAT) — an open standard that combines Lightning payments with API access tokens. Here's how it works behind the scenes:

💬 **1. You send a request**
"Translate this text to French"

🧾 **2. The Ark creates a Lightning invoice**
A small invoice (e.g. 10 sats) with a unique payment hash

3. You pay the invoice

Figure 3: Learn Lightning - Educational page explaining Bitcoin Lightning payments

2.6 Developer API

The Ark AI provides an OpenAI-compatible API for developers at https://arknode.ai/v1/. This allows developers to integrate AI capabilities into their own applications using the standard OpenAI chat completions format. The API supports all 120+ task types, streaming responses, and Lightning payment via the L402 protocol. Additionally, a dedicated file conversion API is available at https://arknode.ai/convert/api/ for programmatic file conversions.

2.7 In-App Wallet System

Users can create an in-app wallet by clicking the "Wallet" button in the navigation bar. The wallet allows users to deposit Bitcoin via Lightning and maintain a balance for paying for tasks. This eliminates the need to create a new Lightning invoice for each task. The wallet system tracks balance, transaction history, and automatically deducts payment for each task.

3.0 Software Source Code

The following sections present key portions of the source code for The Ark AI platform. The software is built primarily in Python using the FastAPI framework, with a JavaScript/HTML5 frontend. The complete codebase includes approximately 4,200 lines of server-side code.

3.1 Main Application Server (server.py) - Imports and Configuration

server.py - Application initialization, imports, and task configuration

```
 """
Bitcoin AI Agent Server
FastAPI server with L402 Lightning payment gating.

Usage:
    python server.py                    # uses config.yaml
    LNBITS_URL=... python server.py     # env overrides
"""

import os
import sys
sys.path.insert(0, os.path.join(os.path.dirname(__file__), "../data"))
from tracking import track_event, get_report
import sys
import time
import json
import hmac
import hashlib
import base64
import logging
import secrets
import sqlite3
import threading
from pathlib import Path
from typing import Any, Optional

import httpx
import yaml
from dotenv import load_dotenv
from fastapi import FastAPI, Request, Response, HTTPException
from fastapi.responses import JSONResponse
from fastapi.responses import FileResponse as _FileResp
from pydantic import BaseModel
from pdf_gen import generate_pdf, PDF_ELIGIBLE_TASKS, cleanup_old_pdfs, PDF_DIR

from workflows import WORKFLOWS
MAX_INPUT_LENGTH = 50000  # 50KB max input
MAX_TASK_INPUT = 100000  # 100KB for research tasks

from llm_tasks import execute_task as _execute_task_raw, estimate_price, TASK_TYPES, text_to_speech, generate_image, edi
from llm_tasks import get_model_for_task, FAST_MODEL, POWER_MODEL, POWER_TASKS
from openai_compat import register_openai_routes
from wallet import register_wallet_routes, wallet_deduct, wallet_balance

def _auto_refund(client_ip, amount_sats, payment_hash, reason="task_failure"):
    """Auto-credit a user when their paid task fails."""
    import json, time
    credits_file = "/opt/bitcoin-agent/data/credits.json"
    try:
        try:
...
[11 more lines]
```

3.2 Streaming Task Endpoint

server.py - /task/stream endpoint handling user requests

```
 @app.post("/task/stream")
async def stream_task(req: TaskRequest, request: Request):
    """Execute a task with streaming response (SSE)."""
    client_ip = request.headers.get("X-Real-IP", request.client.host)

    # Check free prompts
    import hashlib as _hl2
    ua = request.headers.get("user-agent", "")
    al = request.headers.get("accept-language", "")
    fp = "fp:" + _hl2.sha256((ua + "|" + al).encode()).hexdigest()[:16]

    is_free = _check_free_prompt_available(client_ip, req.task) and _check_free_prompt_available(fp, req.task)

    # Check wallet
    wallet_id = (req.params or {}).get("wallet_id") or request.headers.get("x-wallet-id")
    has_wallet = False
    price = 0
    if not is_free:
        # Check credits
        _credits_file = "/opt/bitcoin-agent/data/credits.json"
        has_credit = False
        try:
            import json as _cjson
            with open(_credits_file, "r") as _cf:
                _credits = _cjson.load(_cf)
            pricing_c = CONFIG.get("pricing", {})
            price_c = estimate_price(req.task, req.input, pricing_c)
            if client_ip in _credits and _credits[client_ip].get("credit_sats", 0) >= price_c:
                has_credit = True
                price = price_c
        except:
            pass

        if not has_credit:
            pricing = CONFIG.get("pricing", {})
            price = estimate_price(req.task, req.input, pricing)
```

3.3 File Converter Service (file-converter/server.py)

server.py - File converter initialization and conversion logic

```
 """
The Ark AI -- File Converter API
A universal file conversion service powered by FFmpeg, LibreOffice, and ImageMagick.
"""
import os, uuid, time, asyncio, shutil, json, subprocess, logging
from pathlib import Path
from fastapi import FastAPI, UploadFile, File, Form, HTTPException, Request
from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
from fastapi.staticfiles import StaticFiles
from fastapi.middleware.cors import CORSMiddleware

# PDF conversion tools
try:
    from pdf2docx import Converter as PDFConverter
    HAS_PDF2DOCX = True
except ImportError:
    HAS_PDF2DOCX = False

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("converter")

app = FastAPI(title="The Ark AI File Converter", version="1.0.0")
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_headers=["*"])

UPLOAD_DIR = Path("/opt/file-converter/uploads")
OUTPUT_DIR = Path("/opt/file-converter/converted")
STATIC_DIR = Path("/opt/file-converter/static")
MAX_FILE_SIZE = 50 * 1024 * 1024  # 50MB
FILE_EXPIRY = 3600  # 1 hour

UPLOAD_DIR.mkdir(exist_ok=True)
OUTPUT_DIR.mkdir(exist_ok=True)

# === Supported Conversions ===
CONVERSIONS = {
    # Documents
    "pdf": {"from": ["docx","doc","odt","txt","html","rtf","epub","xlsx","csv","pptx","png","jpg","jpeg","bmp","tiff","g
    "docx": {"from": ["pdf","odt","txt","html","rtf"], "tool": "libreoffice"},
    "txt": {"from": ["pdf","docx","odt","rtf","html"], "tool": "libreoffice"},
    "html": {"from": ["docx","odt","txt","md"], "tool": "libreoffice"},
    "odt": {"from": ["docx","doc","txt","rtf"], "tool": "libreoffice"},
    # Images
    "png": {"from": ["jpg","jpeg","webp","bmp","tiff","tif","gif","svg","heic","ico","avif"], "tool": "imagemagick"},
    "jpg": {"from": ["png","webp","bmp","tiff","tif","gif","svg","heic","ico","avif"], "tool": "imagemagick"},
    "jpeg": {"from": ["png","webp","bmp","tiff","tif","gif","svg","heic","ico","avif"], "tool": "imagemagick"},
    "webp": {"from": ["png","jpg","jpeg","bmp","tiff","gif","svg"], "tool": "imagemagick"},
    "gif": {"from": ["png","jpg","jpeg","webp","bmp","mp4","mov","avi"], "tool": "mixed"},
    "bmp": {"from": ["png","jpg","jpeg","webp","tiff"], "tool": "imagemagick"},
    "tiff": {"from": ["png","jpg","jpeg","webp","bmp"], "tool": "imagemagick"},
    "ico": {"from": ["png","jpg","jpeg","svg"], "tool": "imagemagick"},
...
[11 more lines]
```

3.4 Application Configuration (config.yaml)

config.yaml - Application configuration (sensitive values redacted)

```
 admin:
  token: ark-nav-2026-secure
l402:
  location: https://thenode.it.com
  secret_key: ENV
  token_expiry_seconds: 86400
lightning_address:
  ad_copy: 20
  agent_debug: 100
  agent_design: 200
  agent_prompt_eng: 100
  api_doc: 100
  api_parse: 20
  bug_finder: 100
  changelog_gen: 20
  cicd_gen: 100
  code_explain: 10
  code_review: 100
  color_palette: 20
  commit_msg: 10
  compose_gen: 100
  cron_gen: 10
  data_clean: 20
  description: Bitcoin AI Agent - pay for tasks
  doc_gen: 100
  dockerfile_gen: 20
  domain: thenode.it.com
  email_subject: 20
  enabled: true
  json_csv: 10
  langchain_gen: 200
  langgraph_gen: 200
  max_sats: 100000
  min_sats: 1
  multi_agent_gen: 500
  nginx_gen: 20
  privacy_policy: 200
  readme_gen: 100
  regex_gen: 20
  seo_meta: 20
  sql_optimize: 20
  tech_blog: 200
  terms_gen: 200
  unit_test: 100
  username: agent
llm:
  api_key: [REDACTED]
  base_url: https://api.deepinfra.com/v1/openai
  max_tokens: 8192
  model: Qwen/Qwen3.5-122B-A10B
...
[154 more lines]
```

4.0 Technology Stack

The Ark AI is built using the following technologies:

- Backend Framework:	Python 3.12, FastAPI, Uvicorn
- AI/LLM Engine:	Qwen 3.5-122B-A10B via DeepInfra API
- Image Generation:	OpenAI DALL-E API
- Voice/Speech:	OpenAI Whisper (transcription), OpenAI TTS (text-to-speech)
- Payment System:	Phoenixd (Lightning Node), LNbits (Wallet Management)
- File Conversion:	LibreOffice, ImageMagick, FFmpeg, pdf2docx, PyMuPDF
- Web Server:	Nginx (reverse proxy, SSL termination)
- Frontend:	HTML5, CSS3, JavaScript (vanilla)
- Operating System:	Ubuntu 24.04 LTS
- Hosting:	DigitalOcean Cloud VPS
- SSL/TLS:	Let's Encrypt (certbot)
- Security:	fail2ban, iptables, rate limiting
- Version Control:	Git, GitHub

5.0 Deployment Architecture

The Ark AI runs on a cloud-based virtual private server with the following architecture:

- Nginx serves as the entry point, handling HTTPS termination and routing
- The main AI service (bitcoin-agent) runs on port 8402
- The file converter service runs on port 8500
- LNbits wallet management runs on port 5000
- Phoenixd Lightning node handles payment processing
- All inter-service communication happens over localhost
- External access is restricted via iptables firewall rules
- SSL certificates are automatically renewed via certbot